TÉCNICAS DE PROGRAMACIÓN ORIENTADA A OBJETOS

UNIDAD 1:
Fundamentos de Programación Orientada a Objetos

SESIÓN 2:
Estructura de programas, clases de biblioteca y modificadores de acceso

Dr. Eric Gustavo Coronel Castillo
eric.coronel@upn.pe

MOTIVACIÓN

DESAFIO DEL DIA

Un programa claro es una conversación: si no se entiende leyendo, tampoco se entenderá ejecutando.

MOTIVACIÓN

IDEA FUERZA

Las estructuras entrada-proceso–salida, condicionales, bucles y el uso correcto de librerías hacen que el código “hable” con claridad.

LOGRO DE LA UNIDAD 1

Al finalizar la unidad, el estudiante desarrolla casos aplicando correctamente los principios de programación orientada a objetos, demostrando buenas prácticas de programación y utilizando sistemas de control de versiones para gestionar eficazmente su código.

LOGRO DE LA SESIÓN

El estudiante implementa programas en Java utilizando el escáner para la entrada de datos, aplica sentencias condicionales y de control, y emplea las clases de biblioteca String, Random y Math, respetando los modificadores de acceso en la definición de clases y métodos.

REFLEXIONA

PREGUNTA 1

¿Qué crees que significa que Java sea un lenguaje orientado a objetos? ¿Puedes dar un ejemplo de un «objeto» del mundo real y describir qué características tendría?

PREGUNTA 2

¿Cuál es la diferencia entre una clase y un objeto en programación? ¿Qué rol cumplen los métodos y los atributos dentro de una clase?

REFLEXIONA

R1 — Java y la Orientación a Objetos

Java es un lenguaje de programación orientado a objetos donde "todo es un objeto". Un objeto del mundo real como un **Automóvil** tiene atributos (marca, color, velocidad) y comportamientos (acelerar, frenar). En Java esto se modela con atributos y métodos dentro de una clase.

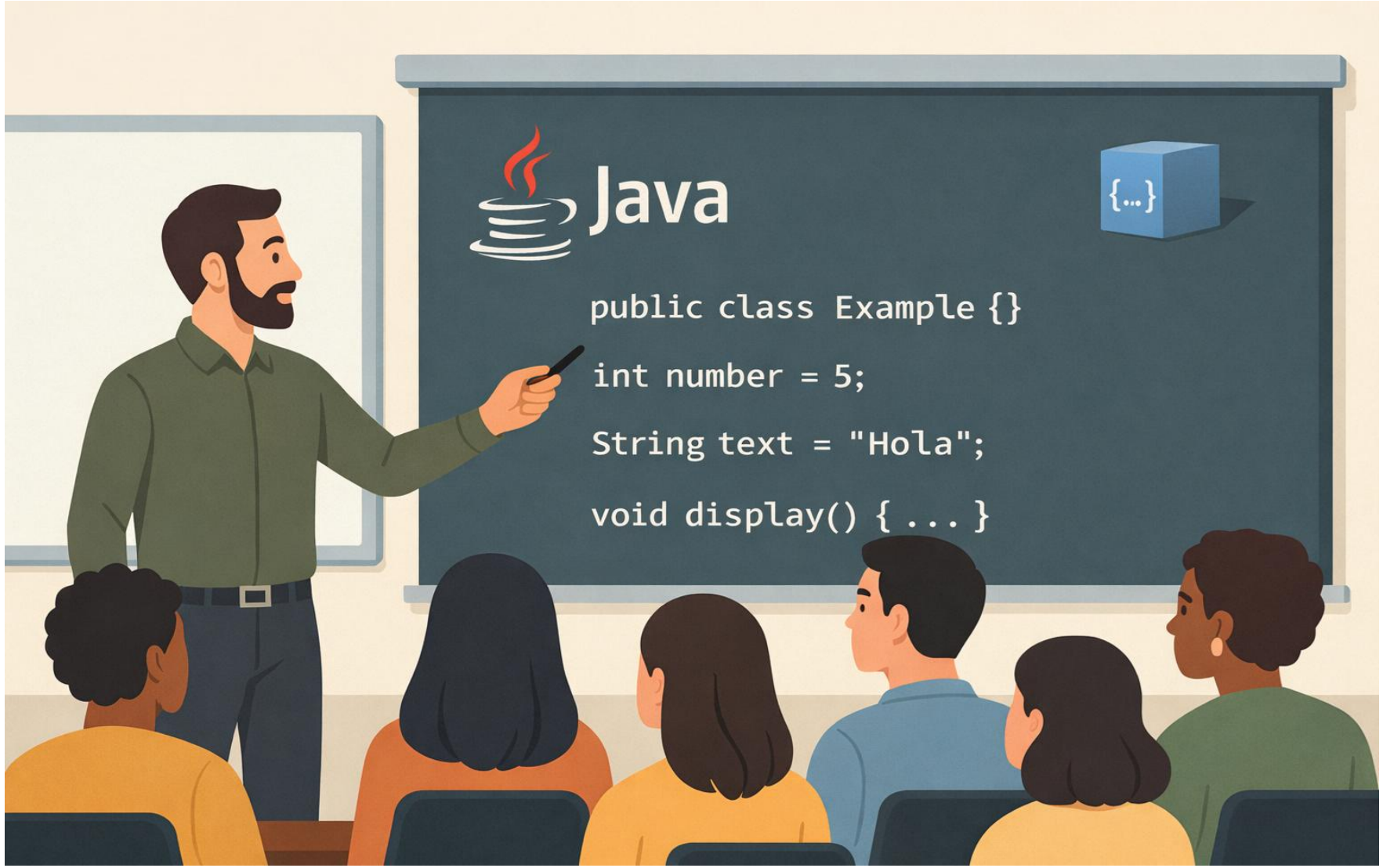
Fuente: Blasco, F. (2020). Programación en Java. Digitalia/UPN. https://digitalia.bibliotecaupn.elogim.com/a/110219

R2 — Clase vs. Objeto; Métodos y Atributos

Una **clase** es la plantilla o molde (p.ej. Automóvil); un **objeto** es una instancia concreta (p.ej. miAuto = new Automóvil()). Los **atributos** representan el estado y los **métodos** el comportamiento del objeto.

Fuente: Vegas Gertrudix, J. M. (2021). Java 17: fundamentos prácticos de programación. eLibro/UPN. https://elibro.bibliotecaupn.elogim.com/es/lc/upnorte/titulos/222664

DESARROLLO


Java
{...}
public class Example {}
int number = 5;
String text = "Hola";
void display() { ... }

TEMARIO

- Principios, evolución, abstracción de datos.
- Ocultamiento y encapsulamiento.
- Necesidad de encapsular datos.
- Clases y Objetos.
- Ciclo De Vida.

ABSTRACCIÓN DE DATOS - POO

Persona 1
Nombre: José
Profesión: Granjero

Persona 4
Nombre: Ricardo
Profesión: Cartero

1 CLASE
12 OBJETOS

1 CLASE: Persona
12 OBJETOS: Instancias de la clase

Persona 12
Nombre: Geraldine
Profesión: Peluquera

¿CÓMO SE RELACIONA A LA PRÁCTICA PROFESIONAL?

- Está muy ligada a la comprensión y modelamiento de las reglas del negocio.
- Por ejemplo, el término “cuenta” podría tener diferentes acepciones de acuerdo al entorno/industria en el que estemos trabajando.

OCULTAMIENTO Y ENCAPSULAMIENTO

- Ocultamiento es la propiedad que permite asegurar que los atributos (y sus estados) de un objeto estén ocultos al exterior.
- Encapsulamiento consiste en declarar dentro de una clase a los atributos con un acceso privado y los métodos con un acceso público.
- Esto conlleva a que para acceder, ya sea para consultar o modificar a los miembros de un objeto, esto deberá realizarse a través de sus métodos públicos.

DEMO EN NETBEANS

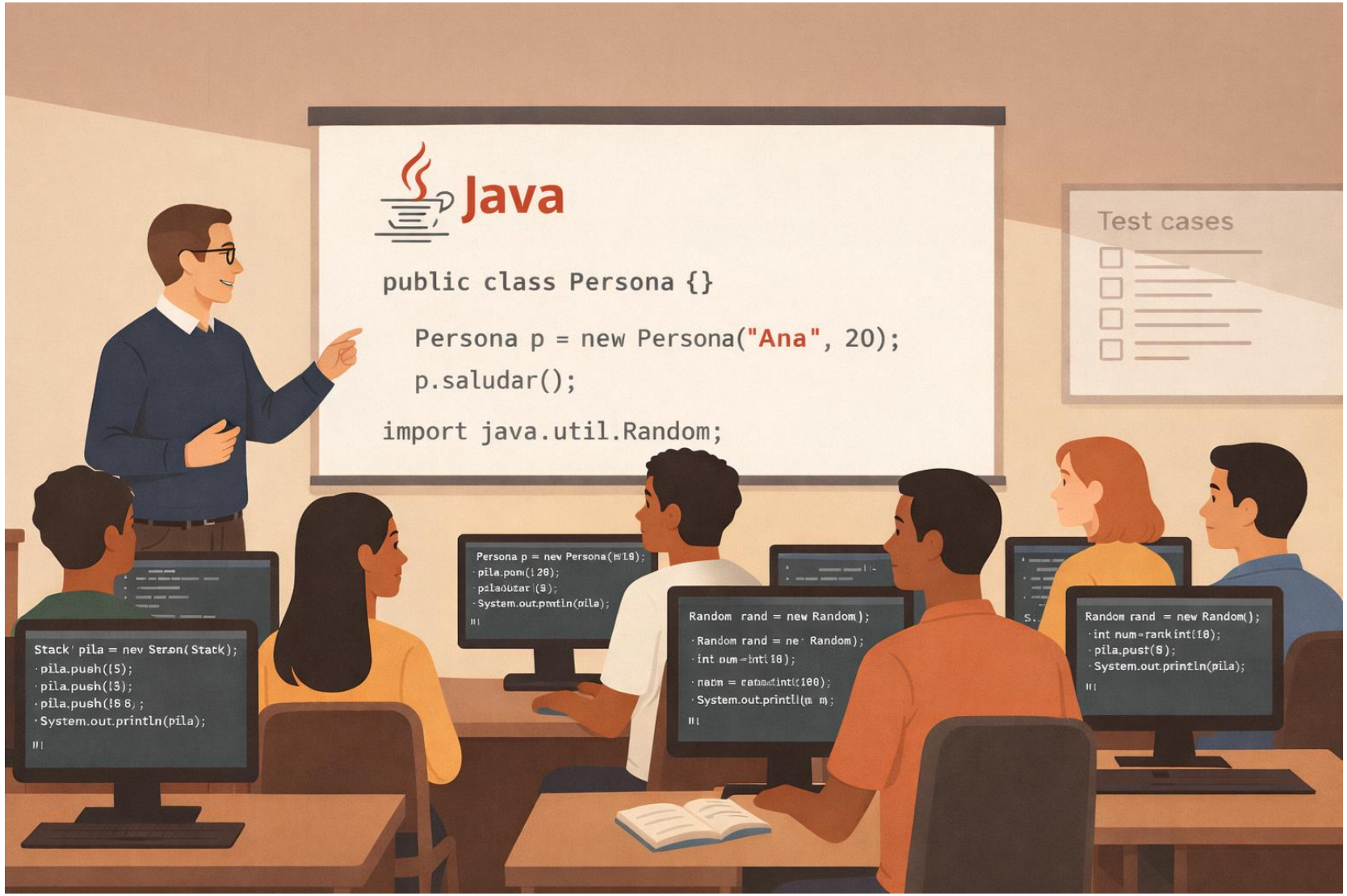
NECESIDAD DE ENCAPSULAR DATOS

- El encapsulamiento de datos permite que los atributos y estados de un objeto sean accesibles a través de métodos públicos.
- Cuando esto no sucede, es decir cuando se establece que los atributos sean públicos, éstos podrían tomar valores inconsistentes, lo cual sería fatal para cualquier sistema.

CONCEPTOS

1. Instancia / objeto
2. Clase
3. Atributos (private)
4. Métodos (public)
5. Métodos set y get
6. Constructor
7. Abstracción
8. Encapsulamiento

ACTIVIDAD EN CLASE



👩‍💻 Ejercicio 1 de 4

🎯 Calculadora de Notas con Scanner y Condicionales

📋 Enunciado

Desarrolla un programa Java que:

1. Solicite al usuario 3 notas (tipo double) usando Scanner.

2. Calcule el promedio.

3. Usando if-else muestre:

- Aprobado (≥ 11)
- Proceso (7 - 10)
- Desaprobado (< 7)

4. Use Math.round() para redondear el promedio.

```java
// Estructura de apoyo (completar)

import java.util.Scanner;
public class Notas {
  public static void main(String[] a) {
    Scanner sc = new Scanner(System.in);
    // TODO: leer 3 notas
    double n1, n2, n3;
    // TODO: calcular promedio
    double prom;
    // TODO: if-else resultado
  }
}
```

👩‍💻 Ejercicio 2 de 4

🎯 Modelado OO: Clase Producto para una Tienda

Diagrama UML (Clase)

Producto

```
- nombre: String
- precio: double
- stock: int
```

```
+ Producto(n,p,s)
+ getNombre(): String
+ getPrecio(): double
+ aplicarDescuento(d)
```

📋 Requerimientos

1. Crea la clase Producto con atributos privados: nombre (String), precio (double), stock (int).
2. Implementa un constructor que inicialice los 3 atributos.
3. Crea getters para cada atributo.
4. Agrega el método `aplicarDescuento(double d)` que reduzca el precio en d%.
5. En el main, crea 2 objetos Producto e imprime sus datos con un bucle for.

👨‍💻 Ejercicio 3 de 4

🎯 Reto: Juego de Adivinar Número con Random y Bucle

📋 Enunciado

Crea un juego interactivo que:

1. Genere un número secreto entre 1 y 100 usando Random.

2. Pida al usuario ingresar un número con Scanner.

3. Dentro de un bucle while, indique: «Muy alto», «Muy bajo» o «¡Correcto!».

4. Cuente los intentos y muéstrolos al ganar.

💡 **BONUS:** Encapsula el juego en una clase «JuegoAdivinar» con un método iniciar().

```
Salida esperada:

=== Juego de Adivinanza ===
Número: 1-100
Tu intento: 45
>> Muy alto

Tu intento: 20
>> Muy bajo

Tu intento: 33
¡Correcto!
Intentos: 3
```

👩‍💻 Ejercicio 4 de 4

🎯 Reto: Menú de opciones

📋 Enunciado

Construya un menú con estas opciones:

1. Calcular el cuadrado de un número.
2. Calcular la raíz cuadrada de un número.
3. Salir.
4. Mostrar mensaje de opción inválida cuando corresponda.

Pistas: use Scanner, switch y métodos de Math.

Estructura de apoyo

```
switch (opcion) {
    case 1:
        System.out.println(Math.pow(n, 2));
        break;
    case 2:
        System.out.println(Math.sqrt(n));
        break;
    case 3:
        System.out.println("Fin");
        break;
    default:
        System.out.println("Opción no válida");
}
```

CONCLUSIONES

CONCLUSIONES

01 La clase es la unidad fundamental de la POO

Encapsula estado (atributos) y comportamiento (métodos). Dominar su estructura es la base para todo desarrollo OO.

02 Scanner + condicionales + bucles = flujo real

Estos tres elementos permiten crear programas interactivos. El control de flujo es la columna vertebral de cualquier algoritmo.

03 Modificadores de acceso = Encapsulamiento

private protege el estado interno; public expone la interfaz. Este principio SOLID garantiza bajo acoplamiento y alta cohesión.

04 String, Math y Random: biblioteca estándar de Java

Reutilizar la JDK evita reinventar la rueda. El código limpio aprovecha las APIs del lenguaje antes de escribir lógica propia.

¿PREGUNTAS O COMENTARIOS?

¿PREGUNTAS?

¡El momento de despejar dudas es ahora!

Participen activamente — todas las preguntas son bienvenidas.

COMPROBACIÓN DEL LOGRO

COMPROBACIÓN DEL LOGRO

Cuestionario en la plataforma

Nivel	Rango
Nivel 4	17 – 20
Nivel 3	13 – 16
Nivel 2	9 – 12
Nivel 1	0 - 8

COMPROBACIÓN DEL LOGRO

Criterio	Nivel 4 (Excelente)	Nivel 3 (Bueno)	Nivel 2 (Regular)	Nivel 1 (Deficiente)
Comprensión conceptual	Demuestra comprensión completa de los conceptos clave.	Comprende la mayoría de los conceptos clave.	Muestra comprensión parcial, con algunas confusiones.	No demuestra comprensión clara de los conceptos.
Aplicación de conocimientos	Aplica correctamente los conceptos en ejemplos o escenarios.	Aplica conceptos con algunos errores menores.	Aplica de forma superficial o con errores evidentes.	No logra aplicar los conceptos o lo hace incorrectamente.
Claridad y precisión en respuestas	Las respuestas son claras, bien estructuradas y precisas.	Las respuestas son mayormente claras, con leves imprecisiones.	Respuestas poco claras o con ideas mal organizadas.	Respuestas confusas, incompletas o incoherentes.
Cobertura del contenido esperado	Responde completamente las 5 preguntas, desarrollando bien cada una.	Responde correctamente 4 de las 5 preguntas.	Responde correctamente 2 o 3 preguntas.	Responde solo 1 pregunta correctamente o no responde.

BIBLIOGRAFIA REFERENCIAL

BIBLIOGRAFIA REFERENCIAL

[BIBLIOGRAFÍA BÁSICA]

Vegas Gertrudix, J. M. (2021). Java 17: fundamentos prácticos de programación. eLibro.
https://elibro.bibliotecaupn.elogim.com/es/lc/upnorte/titulos/222664

[BIBLIOGRAFÍA COMPLEMENTARIA]

Blasco, F. (2020). Programación en Java. Digitalia Hispánica.
https://digitalia.bibliotecaupn.elogim.com/a/110219

[BIBLIOGRAFÍA COMPLEMENTARIA]

Blasco, F. (2020). Programación Java: JDBC y Swing. eLibro.
https://elibro.bibliotecaupn.elogim.com/es/lc/upnorte/titulos/222696

[BIBLIOGRAFÍA COMPLEMENTARIA]

Jiménez de Parga, C. (2021). UML: arquitectura de aplicaciones en Java, C++ y Python. eLibro.
https://elibro.bibliotecaupn.elogim.com/es/lc/upnorte/titulos/222720

[FUENTE OFICIAL]

Oracle Corporation. (2024). The Java™ Tutorials. Oracle Technology Network.
https://docs.oracle.com/javase/tutorial/

GRACIAS

UPN

UNIVERSIDAD
PRIVADA
DEL NORTE